An Approach for Tiling and Fusing Dependent Reductions in Linalg

MLIR Developer Meetup - Oct 2026

Charitha Saumya
Intel MLIR Team

intel.

Tile, Fuse Producers/Consumers, and Repeat

```
// prologue: A + c
%a = linalg.add ins(%A, %cst) outs(%e0)

// GEMM
%c = linalg.matmul ins(%a, %B) outs(%C)

// epilogue: exp
%d = linalg.exp ins(%c) outs(%D)
```

Tile the GEMM
Fuse producers
Fuse consumers



```
%r = scf.forall (%i, %j) = (0, 0) to (M, N)
       step (tM, tN) shared_outs(%o = %D) {
  %At = tensor.extract_slice %A[%i, 0] [tM, K]
  %at = linalg.add ins(%At, %cst) outs(%e0t)
  %Bt = tensor.extract_slice %B[0, %j] [K, tN]
  %Ct = tensor.extract_slice %C[%i, %j] [tM, tN]
  %ct = linalg.matmul ins(%at, %Bt) outs(%Ct)
  %Ot = tensor.extract_slice %o[%i, %j] [tM, tN]
  %dt = linalg.exp ins(%ct) outs(%Ot)
  scf.forall.in_parallel {
    tensor.parallel_insert_slice %dt into
        %o[%i, %j] [tM, tN]
  }
}
```

Tile, Fuse Producers/Consumers, and Repeat

```
// prologue: A + c
%a = linalg.add ins(%A, %cst) outs(%e0)

// GEMM
%c = linalg.matmul ins(%a, %B) outs(%C)

// epilogue: exp
%d = linalg.exp ins(%c) outs(%D)
```

Tile the GEMM
Fuse producers
Fuse consumers



```
%r = scf.forall (%i, %j) = (0, 0) to (M, N)
       step (tM, tN) shared_outs(%o = %D) {
  %At = tensor.extract_slice %A[%i, 0] [tM, K]
  %at = linalg.add ins(%At, %cst) outs(%e0t)
  %Bt = tensor.extract_slice %B[0, %j] [K, tN]
  %Ct = tensor.extract_slice %C[%i, %j] [tM, tN]
  %ct = linalg.matmul ins(%at, %Bt) outs(%Ct)
  %Ot = tensor.extract_slice %o[%i, %j] [tM, tN]
  %dt = linalg.exp ins(%ct) outs(%Ot)
  scf.forall.in_parallel {
    tensor.parallel_insert_slice %dt into
        %o[%i, %j] [tM, tN]
  }
}
```

Great support for tiling along parallel/reduction dims and fusion along parallel dims

How about Dependent Reductions?

```
// R1: reduce along the reduction dim
%m = linalg.reduce {red_kind1}
       ins(%x) outs(%init1)
       dimensions = [1]
// R1's result, broadcast along it
%p = linalg.generic {...}
       ins(%x, %m) outs(%init)
// R2: reduce along the same dim
%s = linalg.reduce {red_kind2}
       ins(%p) outs(%init2)
       dimensions = [1]
```

Tile R2
Fuse producers



```
%s = scf.for %j = 0 to N step tN iter_args(%a) {
  %xt = tensor.extract_slice %x[0, %j] [M, tN]
  // R1 fused at FULL extent: reads all of x
  %m  = linalg.reduce {red_kind1} ins(%x) outs(%i1)
  %pt = linalg.generic {...} ins(%xt, %m) outs(%et)
  %st = linalg.reduce {red_kind2} ins(%pt) outs(%a)
  scf.yield %st
}
```

R2's tile needs R1's result over the *whole* row ⇒ R1 can't be tiled
It re-reads all of `x` in *every* iteration ⇒ bad performance

Settle for Separate Loops

```
// R1: reduce along the reduction dim
%m = linalg.reduce {red_kind1}
       ins(%x) outs(%init1)
       dimensions = [1]
// R1's result, broadcast along it
%p = linalg.generic {...}
       ins(%x, %m) outs(%init)
// R2: reduce along the same dim
%s = linalg.reduce {red_kind2}
       ins(%p) outs(%init2)
       dimensions = [1]
```

Tile R1
Tile R2
No fusion



```
%m = scf.for %j = 0 to N step tN iter_args(%a) {
  %xt = tensor.extract_slice %x[0, %j] [M, tN]
  %mt = linalg.reduce {red_kind1} ins(%xt) outs(%a)
  scf.yield %mt
}
%s = scf.for %j = 0 to N step tN iter_args(%a) {
  %xt = tensor.extract_slice %x[0, %j] [M, tN]
  %pt = linalg.generic {...} ins(%xt, %m) outs(%et)
  %st = linalg.reduce {red_kind2} ins(%pt) outs(%a)
  scf.yield %st
}
```

Better: R1 is computed once, no recompute

Not the best: x is read twice ⇒ costly when the reduction dim is huge

Can we do better? Online Softmax

$$m = \max_j x_j$$

$$s = \sum_j e^{x_j - m}$$

$$\mathrm{softmax}(x)_j = \frac{e^{x_j - m}}{s}$$

$$e^{x_j - m_{\text{new}}} = e^{x_j - m_{\text{old}}} \cdot e^{m_{\text{old}} - m_{\text{new}}}$$

```
m = -inf;  s = 0
for each tile x_t of x:
  m_new = max(m, max(x_t))
  s = s * exp(m - m_new)     // correction
      + sum(exp(x_t - m_new))
  m = m_new
```

One pass over `x`: rescale the running sum whenever the max changes

When/Why does this work?

$$m = \max_j x_j$$

$$\mathrm{softmax}(x)_j = \frac{e^{x_j - m}}{s}$$

$$s = \sum_j e^{x_j - m} = \sum_j \underbrace{e^{x_j}}_{\text{data}} \cdot \underbrace{e^{-m}}_{\text{R1}}$$

Stage $i-1$

$$s_{i-1} = \sum_{j<i} e^{x_j - m_{i-1}} \tag{1}$$

$$= e^{x_1} e^{-m_{i-1}} + e^{x_2} e^{-m_{i-1}} + \cdots + e^{x_{i-1}} e^{-m_{i-1}} \tag{2}$$

Stage i

$$m_i = \max(m_{i-1}, x_i) \tag{3}$$

$$s_{i-1} \cdot \frac{1}{e^{-m_{i-1}}} = e^{x_1} + e^{x_2} + \cdots + e^{x_{i-1}} \tag{4}$$

$$s_{i-1} \cdot \boxed{\frac{e^{-m_i}}{e^{-m_{i-1}}}} = e^{x_1 - m_i} + e^{x_2 - m_i} + \cdots + e^{x_{i-1} - m_i} \tag{5}$$

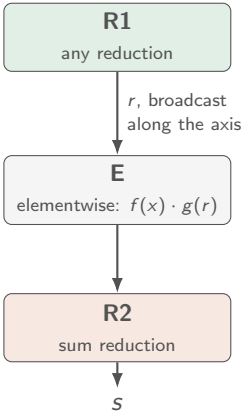
$$s_i = s_{i-1} \cdot e^{m_{i-1} - m_i} + e^{x_i - m_i} \tag{6}$$

R2 is a *sum*, and each summed term is $f(x) \cdot g(\mathrm{R1})$ (multiplicatively separable)
Multiplication distributes over addition $\Rightarrow$ one rescale corrects every term

Legality Conditions

For a reduction chain R1 $\rightarrow$ E $\rightarrow$ R2:

1. **R1** can be *any* reduction
2. **E** (elementwise) uses R1's result *broadcast* along R1's reduction axis
3. E's use of R1 is *multiplicatively separable*: $E(x, r) = f(x) \cdot g(r)$
4. **R2** is a *sum* reduction that reduces over the same axis as R1



Multiplicative Separability

A lightweight *dataflow analysis* over E's body proves $E(x, m) = f(x) \cdot g(m)$

Facts, propagated from inputs to yield

- {IND}: independent of R1
- {ADD}: R1 enters *additively*
- {MUL}: R1 enters *multiplicatively*

Transfer rules

`add, sub`	keep {ADD}
`mul, div`	keep {MUL}
`exp`	{ADD} → {MUL}
`neg` / `abs`	keep all / keep {MUL}
other ops	no facts

Example: softmax, $E = \exp(x - m)$

```
// %x is {IND}, %m is {ADD, MUL}
%d = arith.subf %x, %m    // {ADD}
%e = math.exp %d          // {MUL}
linalg.yield %e           // {MUL}
```

Yield is {MUL} and depends on `%m` ⇒ separable ✓

Accept E only if its yield is {MUL} and depends on R1